Assignment 1 Report: `spireslayer`

CMPE 230 Systems Programming – Spring 2026

İlhan ALTINAY, 2023400021
Melih Efe SÖNMEZ, 2023400174

April 8, 2026

1 Project Overview

- **What the program does:** The program is a **command-line interpreter** which is based on a **Read-Parse-Respond Loop**—until the **exit** command is received. In addition to the exit command, there are two more input line types: **sentence** and **question**. If the input lines are as specified in the description of the program, it gives an output according to input. Otherwise, it produces "INVALID" output.
- **What the persistent state of the program is:** The persistent state of the program keeps various "Ironclad-related" properties of the game such as gold, HP, max HP, deck, codex, exhaust logs. It is stored in the **GameState struct**, which is constantly updated.
- **What design priorities shaped your implementation:** Simplicity, Validation, Modularity, Debugging
  - **Simplicity:** Complex algorithms were not used, and basic coding skills were used by taking the readability account. For example, for loops were used to find elements.
  - **Validation:** Input lines were controlled whether they are as desired format to prevent crashes.
  - **Modularity:** The implementation were divided into distinct functions, structures, and files to work easily as a group.
  - **Debugging:** Comprehensive output messages were coded to determine where the program made mistakes.
  - **Dynamic Memory Allocation:** The memory is managed dynamically in many structures such as deck, relics, codex, etc. This is because the maximum limit of the inputs was not known beforehand, which may require an increase in allocated memory.

2 Interpreter Design

2.1 Input Handling and Parsing

Description of how one input line is processed:

- Firstly, input line is read by fgets function, and the new line character is chomped. Then, the interpreter checks whether input is "Exit". If not, input line enters execute_line function.
- execute_line function tokenizes the line and tries to find a partial match between sentence command type or query command type.

$$\text{Ironclad } \cdots \Rightarrow \text{sentence type}, \ \ldots? \Rightarrow \text{query type}$$

- If it succeeds to find a match one of the types, it looks for an exact match between command types and input line by examining all tokens. Then, some of the functions controls names and integers.

$$\left\{ \begin{array}{r} \text{token}[0] = \text{"Ironclad"} \\ \text{token}[1] = \text{"buys"} \\ \text{token}[2] = \text{"card"} \\ \text{token}[\text{token_count_of_line} - 3] = \text{"for"} \\ \text{token}[\text{token_count_of_line} - 1] = \text{"gold"} \end{array} \right\} \Longrightarrow \begin{array}{l} \text{"Ironclad buys card } \langle C \rangle \\ \text{for } \langle n \rangle \text{ gold"} \end{array}$$

- After all this process, it produces an "INVALID" if execute_line-function returns 0. This case happens when;
  - partial match could not be found,
  - exact match could not be found
  - exact match is found but the name or integer constraints give an error,

2.2 State Representation

Game state is stored in struct, and within this struct some of the parts of it are also a struct.

- **Gold, HP, Floor, Current Room:** These variables are stored directly in the GameState struct.
- **Cards and Upgrades:** These variables are stored as a dynamic array by a pointer to an array of structs. The fullness of the arrays is checked by deck_count and deck_capacity variables which are in GameState Struct.
- **Relics and Potions:** These variables are also stored as a dynamic array separately by pointers to arrays of structs with different variables.

- **Codex Effectiveness Notes, Defeated Counts:** These are similar to previous two points.
- **Exhaust Tags:** This one stores array of strings. So, it has a pointer to a pointer.

The reason behind using dynamic arrays is that there is not a maximum input lines. So, the sizes of arrays are not predictable, and dynamic memory allocation provides a resilience to the program. Some variables are not stored dynamically due to given constraints such as maximum line limit, or they do not need reallocation such as integers and fixed-size arrays.

2.3 Command Execution Flow

Execution Flow Example: Ironclad buys card <C> for <n> gold

- **Which function processes the command:** As described in 2.1, exact match is found in execute_line function. Then, integer and name validity is checked. extract_name function is called, if they are valid. Otherwise, it returns 0, which means "INVALID" output.
- **What checks are performed - What state changes occur:** After extracting name, execute_buy_card function is called, and the gold amount of the current state is checked as a first step. If there exist enough gold, it decides whether reallocation is needed by comparing deck_capacity and deck_size, and realloc function is called to enlarge the deck capacity if it is necessary. After that, gold amount is deducted of the state, and new memory is allocated for the naming of the card by calling my_strdup function. Then, the card attributes are assigned such as is_upgraded, and is_exhaust is determined by check_if_exhaust function, which searches global exhaust cards. As a final step, deck_count is incremented by one.
- **What output is produced:** If the gold amount is enough to buy the card, it gives "≫ Card added: <card-name> ". Otherwise, it produces "≫ Not enough gold" output.

3 Implementation Details

While developing the project, especially during sentence and query commands keeping data consistency required a careful design process. Most difficult and attention demanding parts are given below:

- **Grouping Before Output And Lexicographic Sequencing:** Specifically for "Deck?" and "Potions?" queries, it was necessary to count and display cards with the same name in the deck as a single entry, rather than printing them separately. To solve this problem, we created a temporary structure that holds unique cards and their counts. First we group the cards in the deck and transferred them to this structure, then sorted the cards using classic Bubble sort. Unlike the "Potion?"

query, the sorting process in the "Deck?" query is prepared with a two comparisons, not just by name, but also that base version comes before the upgraded version among cards with the same name.

- **Fighting Logic and Dynamic Card Consumption:** Application of "Ironclad fights enemy" command, includes the most complicated edge cases in the project. If "Ironclad" wins, potions that is effective against the enemy and cards marked "exhaust" had to removed one copy from the deck. Helper functions like "remove_card" and "remove_potion" have been designed to prevent array shifting and memory leaks.

- **Constraints and Dynamic Allocation:** Game rules limits the potions belt with 3 slots, however theoretically structures like deck, relics, and codexs can grow infinitely. For fixed capacity like potion belt, limits are predefined. On the other hand, for those that can grow indefinitely, initial capacities are defined when the program has started and in cases where the limit is exceeded, the size is doubled using "realloc". With the help of this, large inputs will not bother and will not give Segmentation Fault.

3.1 Representative Code Snippet

The code snippet below is the function that runs when a new card is added to the player's deck. It is important part of the project in terms of dynamic memory management and data reliability.

```
void execute_gain_card(GameState *state, const char *card_name) {
    if (state->deck_count >= state->deck_capacity) {
        state->deck_capacity *= 2;
        state->deck = realloc(state->deck, state->deck_capacity *
            sizeof(CardEntry));
    }

    state->deck[state->deck_count].name = my_strdup(card_name);
    state->deck[state->deck_count].is_upgraded = 0;
    state->deck[state->deck_count].is_exhaust = check_if_exhaust(state,
        card_name);
    state->deck_count++;

    printf("Card added: %s\n", card_name);
}
```

- **What the code does:** This functions adds the card to deck. First it checks whether the current capacity is reached or not; if reached, by using `realloc` capacity is doubled. Then it copies the card name to a newly allocated memory area using `my_strdup`. Afterwards, it checks whether the card is in exhaust list or not; if yes, `is_exhaust` is true and will increase deck count.

- **Why it is important:** This shows how dynamic inventory management works in project. In test scenarios with undefined limits like stress tests, the program can safely store an unlimited number of cards without exceeding memory limits.

- **What mistakes could occur here:** Two critical errors can occur:
  1. *Skipping capacity control:* If exceeded, *Segmentation Fault* can occur.
  2. *Missing is_upgraded zero:* If the `is_upgraded` value hadn't been given a zero; due to random garbage data in memory, the card could be falsely detected as "upgraded".
- **Why your implementation works correctly:** We used a proactive capacity expansion strategy (`realloc`) to prevent potential memory overflows. With the help of `my_strdup` function, strings are stored in secure memory areas, preventing "dangling pointer" issues.

4 Testing and Validation

Project's full compliance with the rules of the game and its stability in memory management have been tested with multiple test cases.

4.1 Testing Strategy

The test methods below were used to test the project's accuracy:

- **Provided Grader:** "Make grade" command and test files provided within the Github repo have been used as the primary criterion to ensure compliance with the general format and output accuracy.
- **Custom Input Files:** Special large test files were created that pushed the system to its limits. These files tested situations such as adding and deleting dozens of different items simultaneously, proving that dynamic memory allocation (`malloc`/`realloc`) works successfully.

4.2 Sample Interaction

The following brief interaction is demonstrating how the program handles valid inputs, invalid command formats, and inventory constraints.

```
Ironclad gains 005 gold
>> INVALID
Ironclad buys potion Weak Potion for 10 gold
>> Not enough gold
Ironclad gains relic Anchor
>> Relic obtained: Anchor
Ironclad gains relic Anchor
>> Already has relic: Anchor
Total gold?
>> INVALID
Exit
```

4.3 Edge Cases

To measure the project's reliability following edge cases have been specifically tested and gave successful outputs:

- **Malformed Commands:** Inputs that violate parsing rules, such as `Total gold?` which lacks a space before the question mark or `Ironclad gains card` missing name parameter, have been rejected.
- **Extra Tokens:** Extra words like `Exit now` added to the end of a valid command are rejected.
- **Missing Items:** If an attempt is made to delete or sell an item that is not in inventory, "Not found" warnings are generated.
- **Duplicate Relics:** Attempts to gain or purchase a relic that is already owned are prevented.
- **Full Potion Belt:** When an attempt was made to obtain a new potion while the potion belt capacity (3) is full, the system rejects the operation to prevent memory overflow..
- **Sorting Behavior in Queries:** In the 'Deck ?' and 'Potions ?' queries, items are ordered lexicographically. For cards with the same name, the algorithm ensures that the "Base" version appears before the "Upgraded" ones.

5 Discussion of Course Concepts

5.1 Data Representation and Memory Organization

The game's current state is encapsulated within structure instead of global variables and passed to functions via pointers '`GameState *state`'. This 'pass-by-reference' prevents unnecessary copying of large structures in function calls, thus saving memory. Furthermore by using nested structures, such as storing '`CodexCounter`' arrays within '`CodexEntry`', organized memory is achieved.

5.2 Strings, Arrays, and Pointers

We used string manipulation and pointers in many places in the project. We wrote a helper functions named '`my_strdup`' which applies '**Deep Copy**' to strings to prevent words from parser being lost or corrupted.

5.3 Memory Management Choices

In this project, dynamic memory management with malloc and realloc is preferred over static arrays Since they lead to a segmentation fault when the capacity is exceeded, lists are initially initialized with a small capacity.

5.4 Robustness and Debugging

While debugging the project, biggest difficulty we have to overcome occur when deleting cards after the fights. When we first start if card was removed from the deck and the array was shifted, the memory holding the name of the removed card was not released, leading to a leak. Manual deletion also created a double free risk due to floating-point sequences. This confusion was solved by writing '`remove_card`' and '`remove_potion`' functions, which handle the deletion and memory cleanup.

5.5 Limitations and Tradeoffs

While the current design appears secure in terms of memory, it does contain trade-offs because of certain data structure choices:

- **Array Shifting Cost:** Decks are stored as sequences of memory blocks. When a card is removed from the deck, shifting all the cards behind it one step forward causes $O(N)$ time complexity which can take so much time if number of cards that are removed is large.
- **Sorting Algorithm:** The queries use *Bubble sort*. While this is sufficient considering the average given deck size is small, in a theoretical bounds test with many cards, using *Merge sort* instead of this algorithm, which has an average size of $O(N^2)$, could provide a performance improvement.

6 Conclusion

- **Strongest part of the design:** Dynamic memory allocation is the strongest part of the program since it provides resilience against unknown large number of input lines. Additionally, tolerance of the parsing mechanism to space characters, and determining whether the input line has correct form are the other robust attributes of the command-line interpreter.
- **Improvements we would make with more time:** The first job is change of searching algorithm from linear searching - whose **time complexity** is $O(N)$ - to another searching algorithm - whose time complexity is $O(1)$ - by using another data structure whose find function time complexity are $O(1)$ such as "hash table". Furthermore, long if-else chains to route commands should be implemented by **Command Dispatcher** architecture.

AI Usage Statement

- **Tool Name:** Gemini 3 Flash, Gemini 3.1 Pro
- **How it was used:**
  For the report: grammar correction, LaTeX code formatting, concept searching (simplicity, validation, etc.).

For the coding section: understanding the assignment, brainstorming, code formatting.

- We wrote this report by ourselves considering our implementation.